23CSE202 DATABASE MANAGEMENT SYSTEMS
S3 B.Tech CSE
Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

3. Database Design Document

Group No : A5

Team Members :

Roll No	Student Name
AM.SC.U4CSE25057	Vaishnav Venu
AM.SC.U4CSE25060	Praful Singh
AM.SC.U4CSE25063	C Lasya
AM.SC.U4CSE25066	Prathijna Mahalingan

Project Title : Hotel Room Booking and Hospitality Management System

3.1. Detailed description of the project

The Hotel Room Booking and Hospitality Management System is a web-based application designed to manage hotel room reservations and daily hotel operations efficiently.

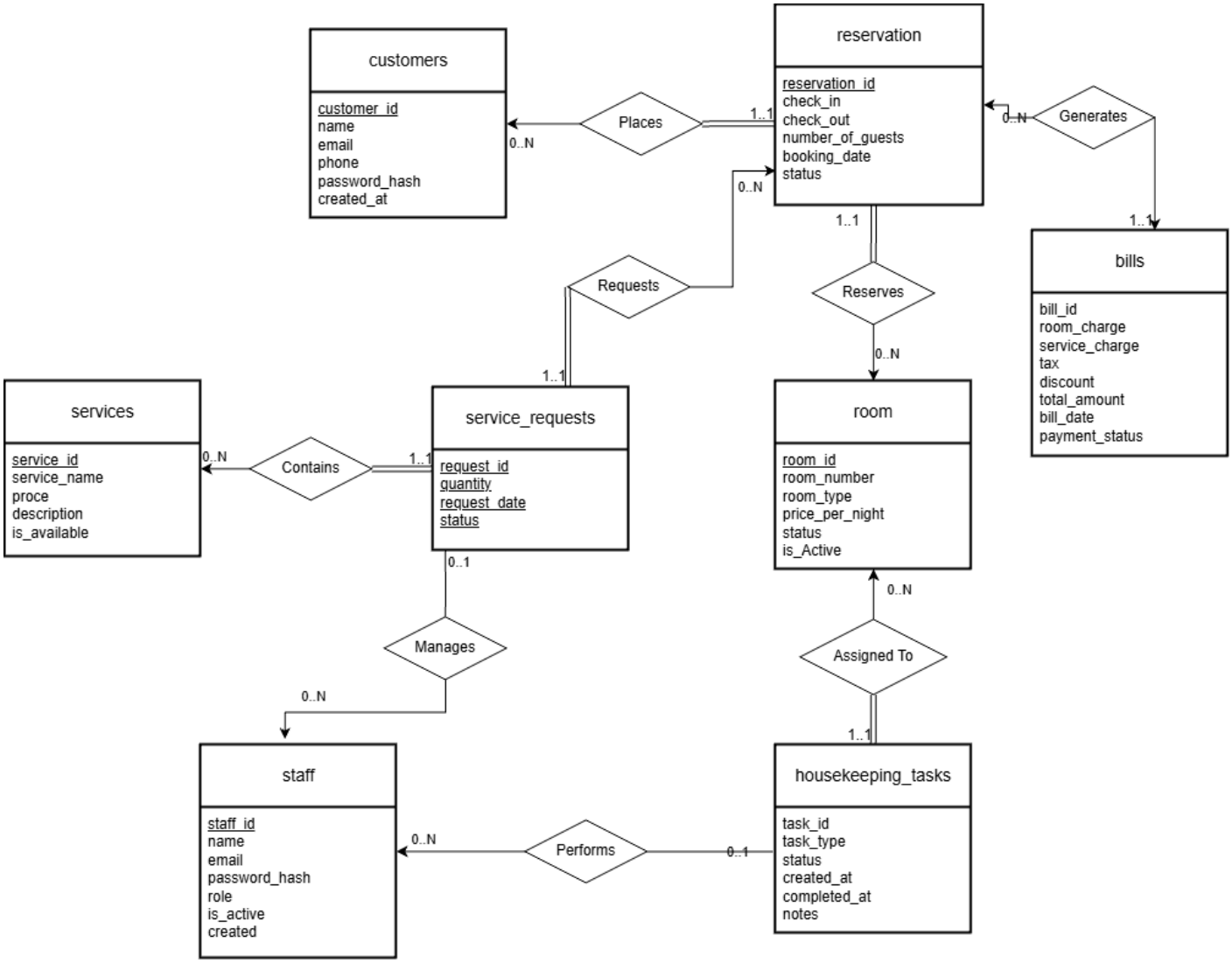
The system uses Python for the backend, PostgreSQL for database management, and a web-based frontend.

Customers can register, log in, check room availability based on their required dates, view room details and prices, and make or cancel reservations. Reception staff can manage customer bookings, check guests in and out, monitor room availability, and handle service requests. The system maintains customer, room, and reservation information in a centralized database.

The system reduces manual work and prevents errors such as double bookings by validating room availability before confirming reservations.

It also provides organized access to booking and room information, making hotel operations faster, more accurate, and easier to manage.

3.2. ER Diagram



3.3 Relational Schema

- customers (customer_id, name, email, phone, password_hash, created_at)
- staff (staff_id, name, email, password_hash, role, is_active, created_at)
- rooms (room_id, room_number, room_type, price_per_night, status, is_active)
- services (service_id, service_name, price, description, is_available)
- reservations (reservation_id, customer_id, room_id, check_in, check_out, number_of_guests, booking_date, status)
- service_requests (request_id, reservation_id, service_id, quantity, request_date, status)
- bills (bill_id, reservation_id, room_charge, service_charge, tax, discount, total_amount, bill_date, payment_status)

- housekeeping_tasks (task_id, room_id, task_type, status, created_at, completed_at, notes)

3.4 Normalization and Functional dependency analysis

Functional Dependencies and their significance:

customers

FD1: customer_id → {name, email, phone, password_hash, created_at}

FD2: email → {customer_id, password_hash, created_at}

rooms

FD1: room_id → {room_number, room_type, price_per_night, status, is_active}

FD2: room_number → {room_id, room_type, price_per_night, status, is_active}

staff

FD1: staff_id → {name, email, password_hash, role, is_active, created_at}

FD2: email → {staff_id, password_hash, role, is_active, created_at}

reservations

FD1: reservation_id → {customer_id, room_id, check_in, check_out, number_of_guests, booking_date, status}

FD2: {room_id, check_in} → {reservation_id, customer_id, check_out, number_of_guests, booking_date, status}

bills

FD1: bill_id → {reservation_id, room_charge, service_charge, tax, discount, total_amount, bill_date, payment_status}

FD2: reservation_id → {bill_id, room_charge, service_charge, tax, discount, total_amount, bill_date, payment_status}

services

FD1: service_id → {service_name, price, description, is_available}

service_requests

FD1: request_id → {reservation_id, service_id, quantity, request_date, status}

housekeeping_tasks

FD1: task_id → {room_id, task_type, status, created_at, completed_at, notes}

Here, in all the tables, the functional dependencies with the left hand side as the primary key, directly hold as they can uniquely identify a record in the table. Similarly, {email} in customers, {room_number} in room, {email} in staff, {room_id, check_in}in

reservations, {reservation_id} in bills are candidate keys, as they cover all attributes in the relation with zero proper subsets holding superkey status; thus the functional dependency holds.

As candidate key is minimal super key, by attribute closure:

Table 1: customers

Candidate Keys: {customer_id}, {email}

Proof 1 (Testing X = {customer_id}):

X^+ = {customer_id}

Apply FD1 → X^+ = {customer_id, name, email, phone, password_hash, created_at} = R

Proof 2 (Testing X = {email}):

X^+ = {email}

Apply FD2 → X^+ = {email, customer_id, password_hash, created_at}

Apply FD1 via customer_id → X^+ = R

Table 2: rooms

Candidate Keys: {room_id}, {room_number}

Proof 1 (Testing X = {room_id}): X^+ = R through its FD1

Proof 2 (Testing X = {room_number}): X^+ = R through its FD2

Table 3: staff

Candidate Keys: {staff_id}, {email}

Proof 1 (Testing X = {staff_id}): X^+ = R through FD1

Proof 2 (Testing X = {email}): X^+ = R through its FD2 and FD1

Table 4: bills

Candidate Keys: {bill_id}, {reservation_id}

Proof 1 (Testing X = {bill_id}): X^+ = R through FD1

Proof 2 (Testing X = {reservation_id}): X^+ = R through FD2

Table 5: reservations

Candidate Keys: {reservation_id}, {room_id, check_in} *(Composite Candidate Key)*

Proof 1 (Testing X = {reservation_id}): X^+ = R via FD1

Proof 2 (Testing X = {room_id, check_in}):

X^+ = {room_id, check_in}

Apply FD2 → X^+ = {room_id, check_in, reservation_id, customer_id, check_out, number_of_guests, booking_date, status} = R

Tables 6: services -- Candidate Key: {service_id}

Table 7: service_requests -- Candidate Key: {request_id}

Table 8: housekeeping_tasks -- Candidate Key: {task_id}

3.4.3 Normalization Demonstration (1NF to BCNF)

For the candidate keys:

- **Prime Attributes:** customer_id, email, room_id, room_number, staff_id, reservation_id, check_in, bill_id, service_id, request_id, task_id
- **Non-Prime Attributes:** name, phone, password_hash, created_at, room_type, price_per_night, status, is_active, role, check_out, number_of_guests, booking_date, room_charge, service_charge, tax, discount, total_amount, bill_date, payment_status, service_name, price, description, is_available, quantity, request_date, task_type, completed_at, notes

1. First Normal Form (1NF)

- All fields must contain atomic attributes.
    - customers: customer_id, name, email, phone, password_hash, created_at all store single atomic primitive values.
    - rooms: room_id, room_number, room_type, price_per_night, status, is_active contain atomic values.
    - staff: staff_id, name, email, password_hash, role, is_active, created_at contain atomic values.
    - reservations: reservation_id, customer_id, room_id, check_in, check_out, number_of_guests, booking_date, status contain atomic values.
    - bills: bill_id, reservation_id, room_charge, service_charge, tax, discount, total_amount, bill_date, payment_status contain atomic values.
    - services: service_id, service_name, price, description, is_available contain atomic values.
    - service_requests: Normalized into discrete scalar rows (request_id, reservation_id, service_id, quantity, request_date, status) rather than storing an array of requested services inside reservations.
    - housekeeping_tasks: Normalized into discrete scalar rows (task_id, room_id, task_type, status, created_at, completed_at, notes) rather than keeping multi-valued log strings inside rooms.

  Thus, satisfies 1NF.

2. Second Normal Form (2NF)

- To be in 2NF, 1NF should ne in 1NF and zero partial dependencies (no non-prime attribute may depend on a proper subset of any candidate key).
    - Single-Attribute Key Tables (customers, rooms, staff, bills, services, service_requests, housekeeping_tasks): Keys consist of a single attribute (customer_id/email, room_id/room_number, staff_id/email, bill_id/reservation_id, service_id, request_id, task_id). Partial dependencies are mathematically impossible.
    - Composite Key Table (reservations with candidate key {room_id, check_in}): Non-prime attributes (reservation_id, customer_id, check_out, number_of_guests, booking_date, status) depend on the full composite occurrence {room_id, check_in}. None of them depend solely on room_id alone or check_in alone.

  Thus, 2NF is satisfied.

3. Third Normal Form (3NF)

- 2NF should be satisfied and zero transitive dependencies should be there (no non-prime attribute determines another non-prime attribute).
    - customers: customer_id → {name, phone, password_hash, created_at}. Non-prime attribute phone does not determine name (phone ↛ name).
    - rooms: room_id → {room_type, price_per_night, status, is_active}. Non-prime attribute status does not determine price_per_night (status ↛ price_per_night).
    - staff: staff_id → {name, role, is_active, created_at}. Non-prime attribute role does not determine name (role ↛ name).
    - reservations: reservation_id → {customer_id, room_id, check_in, check_out, number_of_guests, booking_date, status}. Foreign keys customer_id and room_id are non-prime attributes here, but they do not determine other non-prime fields like check_out or status (customer_id ↛ check_out).
    - bills: bill_id → {reservation_id, room_charge, service_charge, tax, discount, total_amount, bill_date, payment_status}. Non-prime attribute room_charge does not determine tax (room_charge ↛ tax).
    - services: service_id → {service_name, price, description, is_available}. Non-prime attribute price does not determine description (price ↛ description).
    - service_requests: request_id → {reservation_id, service_id, quantity, request_date, status}. Non-prime attribute quantity does not determine request_date (quantity ↛ request_date).
    - housekeeping_tasks: task_id → {room_id, task_type, status, created_at, completed_at, notes}. Non-prime attribute task_type does not determine notes (task_type ↛ notes).

Thus, this satisfies 3NF.

4. Boyce-Codd Normal Form (BCNF)

For every non-trivial functional dependency X → Y, the determinant X must be a superkey.

Considering the functional dependencies, across all tables, there are no functional dependencies driven by non-key attributes. Every single determinant (X) acts as either a single or composite Candidate Key. Therefore, the entire schema successfully satisfies the requirements for Boyce-Codd Normal Form (BCNF).